MODULE 3

SUPERVISED MACHINE LEARNING FOR BUILDING AND DEPLOYING MODELS

MLFP — ML Foundations for Professionals — Terrene Open Academy

"Data is more important than models. But the wrong model can still ruin perfect data."

WHAT YOU WILL LEARN

BY THE END OF MODULE 3

- **Engineer features** from domain knowledge and select the best ones
- **Explain bias-variance** tradeoff and apply regularisation
- **Train the complete model zoo** — SVM, KNN, trees, forests, boosting
- **Evaluate honestly** — the right metric for the right problem
- **Interpret predictions** with SHAP, LIME, and fairness metrics
- **Deploy to production** with registry, drift monitoring, and DataFlow

THREE DEPTH LAYERS

FOUNDATIONS — Everyone follows. Plain language, analogies.

THEORY — Technical depth. Derivations, formulas, code.

ADVANCED — Paper references, edge cases for experts.

A banker and a data scientist sit in the same classroom. Both leave having learned something new.

YOUR JOURNEY: 8 LESSONS

#	Lesson	You Will Be Able To...
3.1	Feature Engineering & Selection	Engineer domain features, prevent leakage, select the best
3.2	Bias-Variance & Regularisation	Diagnose overfitting, apply L1/L2/ElasticNet, cross-validate
3.3	The Complete Model Zoo	Train SVM, KNN, Naive Bayes, trees, forests
3.4	Gradient Boosting Deep Dive	Master XGBoost, LightGBM, CatBoost
3.5	Evaluation, Imbalance & Calibration	Choose the right metric, handle imbalance, calibrate
3.6	Interpretability & Fairness	Explain predictions with SHAP/LIME, measure fairness
3.7	Workflow Orchestration	Build ML workflows, search hyperparameters, register models
3.8	Production Pipeline	Persist to DataFlow, monitor drift, create model cards

KAILASH ENGINES — CUMULATIVE MAP

Module	Engines	Purpose
M1	DataExplorer, PreprocessingPipeline, ModelVisualizer	Explore, clean, visualise
M2	ExperimentTracker, ModelVisualizer	Statistics, regression, hypothesis testing
M3	FeatureEngineer, FeatureStore, TrainingPipeline, AutoMLEngine, HyperparameterSearch, ModelRegistry, EnsembleEngine, WorkflowBuilder, DataFlow, DriftMonitor	Engineer, train, evaluate, deploy, monitor

10 new engines in one module. This is where the Kailash ML lifecycle comes together — from raw features to monitored production models.

WHERE WE ARE

MODULE 1 — DATA PIPELINES

- Polars for data manipulation
- `DataExplorer` for automated profiling
- `PreprocessingPipeline` for cleaning
- `ModelVisualizer` for charts

MODULE 2 — STATISTICS & REGRESSION

- Probability, distributions, Bayesian thinking
- Hypothesis testing and confidence intervals
- Linear regression as first supervised model
- `ExperimentTracker` for reproducibility

Module 3 builds on both: M1 gave you data fluency. M2 gave you statistical foundations and your first model. Now we build the full ML pipeline — from feature engineering to production deployment.

LESSON 3.1

FEATURE ENGINEERING, ML PIPELINE, AND FEATURE SELECTION

"Data is more important than models — and features are the language data speaks."

THE ML PIPELINE

Data Ingestion → Preprocessing → Feature Engineering → Model Selection → Training & Evaluation → Hyperparameter Tuning → Deployment

This module covers every stage. Today: the first three.

Key insight: The pipeline is not linear. You iterate. Bad evaluation sends you back to features. Drift in production sends you back to retraining. Think of it as a cycle, not a line.

STATISTICS VS MACHINE LEARNING

STATISTICS (MODULE 2)

- Explain the past
- "Which variables are significant?"
- p-values, confidence intervals
- Interpretability is paramount
- Small-to-medium data

MACHINE LEARNING (MODULE 3)

- Predict the future
- "What will happen next?"
- Test error, cross-validation
- Predictive accuracy is paramount
- Medium-to-large data

In practice, you need both. Statistics helps you understand; ML helps you act. M2's regression was a bridge between the two worlds.

DATA > MODELS > HYPERPARAMETER TUNING

The hierarchy of ML performance improvement:

1. **Better data and features** — largest impact, 10x improvement potential
2. **Better model choice** — moderate impact, 2-5x improvement
3. **Better hyperparameters** — smallest impact, 1.1-1.5x improvement

> **Common mistake:** Spending days tuning XGBoost hyperparameters when a single well-engineered feature would have improved the model more than all tuning combined.

Example: For HDB price prediction, geocoding an address to latitude/longitude (domain knowledge) is worth more than switching from Random Forest to XGBoost.

TYPES OF ENGINEERED FEATURES

Type	Example (HDB)	Why It Helps
Temporal	Lag price, rolling 3-month mean, month, quarter	Captures trends and seasonality
Interaction	floor_area × storey_range	Non-linear relationships
Polynomial	floor_area²	Diminishing returns on size
Domain	distance_to_mrt, school_count_1km	Domain knowledge encodes causal factors
Aggregation	avg_price_in_town	Neighbourhood context

FEATURE LEAKAGE — THE SILENT KILLER

Leakage: Using information at training time that would NOT be available at prediction time. Results look perfect in testing but fail catastrophically in production.

Common leakage patterns:

- **Target leakage:** A column derived from the target (e.g., "loan_was_approved" to predict "loan_default")
- **Temporal leakage:** Using future data to predict the past (e.g., next month's sales to predict this month)
- **Preprocessing leakage:** Fitting scaler on full dataset before splitting train/test

Prevention rule: For every feature, ask: "Would I have this value at the moment I need to make the prediction?" If no, drop it.

THEORY

FEATURE SELECTION: THREE FAMILIES

FILTER METHODS

Score each feature independently. Fast but ignores feature interactions.

- Mutual information
- Chi-squared test
- Correlation threshold

WRAPPER METHODS

Evaluate subsets by training models. Accurate but expensive.

- Forward selection
- Backward elimination
- RFE (Recursive Feature Elimination)

EMBEDDED METHODS

Selection built into training. Best of both worlds.

- L1 (Lasso) sparsity
- Tree-based importance
- ElasticNet

KAILASH BRIDGE: FEATUREENGINEER & FEATURESTORE

FEATUREENGINEER

Generates and selects features automatically. Supports filter, wrapper, and embedded methods through a unified API.

```
from kailash_ml import FeatureEngineer

engineer = FeatureEngineer()
engineer.fit(df, target="resale_price")
selected = engineer.select_features(method="embedded", threshold=0.01)
transformed = engineer.transform(df)
```

FEATURESTORE

Versioned feature storage with point-in-time correctness — prevents temporal leakage by design.

```
from kailash_ml import FeatureStore

store = FeatureStore()
store.register(df, entity="flat", timestamp_col="month")
```

LESSON 3.1 SUMMARY

- **ML pipeline:** Data → Preprocessing → Features → Model → Evaluate → Tune → Deploy
- **Feature hierarchy:** Data > Models > Hyperparameters
- **Feature types:** Temporal, interaction, polynomial, domain, aggregation
- **Leakage:** Time-travel test every feature
- **Selection:** Filter (fast), Wrapper (accurate), Embedded (practical)
- **Engines:** `FeatureEngineer`, `FeatureStore`

> **Exercise 3.1:** Engineer features for HDB price prediction (geocode, temporal, interactions). Apply 3 selection methods. Compare which features each method selects.

LESSON 3.2

BIAS-VARIANCE, REGULARISATION, AND CROSS-VALIDATION

"Every model makes a tradeoff between being too simple and too complex. The art is finding the sweet spot."

BIAS AND VARIANCE — THE DARTS ANALOGY

Low Bias, Low Var
IDEAL

High Bias, Low Var
CONSISTENTLY WRONG

Low Bias, High Var
RIGHT ON AVERAGE

Bias = how far the centre of your throws is from the bullseye. **Variance** = how spread out your throws are.

BIAS-VARIANCE DECOMPOSITION

Target: $y = f(x) + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Our model $\hat{f}(x)$ is trained on random sample D.

STEP 1: EXPECTED PREDICTION ERROR

$$\text{EPE}(x) = \mathbb{E}_{D,\varepsilon}\left[(y - \hat{f}(x))^2\right]$$

STEP 2: SUBSTITUTE AND ADD/SUBTRACT THE EXPECTED PREDICTION

$$= \mathbb{E}\left[\big(f(x) - \mathbb{E}[\hat{f}] + \mathbb{E}[\hat{f}] - \hat{f} + \varepsilon\big)^2\right]$$

bias variance noise

STEP 3: CROSS TERMS VANISH — FINAL RESULT

$$\text{EPE}(x) = (f(x) - \mathbb{E}[\hat{f}])^2 + \mathbb{E}[(\hat{f} - \mathbb{E}[\hat{f}])^2] + \sigma^2$$

Bias^2 Variance Noise

THE THREE TERMS

Term	What It Measures	You Control It By
Bias^2	Systematic error — model's average is off-target	More complex model, better features
Variance	Sensitivity to training set — predictions fluctuate	Simpler model, more data, regularisation, ensembles
σ^2	Irreducible randomness in the data	Nothing — this is the floor

The tradeoff: Reducing bias (more complex model) increases variance. Reducing variance (simpler model) increases bias. Regularisation is the tool that navigates this tradeoff.

REGULARISATION — CONTROLLING COMPLEXITY

Add a **penalty** to the loss function that discourages large coefficients:

$$\mathcal{L}_{\text{regularised}} = \mathcal{L}_{\text{data}} + \lambda \cdot \Omega(\boldsymbol{\beta})$$

λ controls the strength of regularisation. Higher λ = simpler model.

L1 (LASSO)

$$\Omega = \sum_{i=1}^{p} |\beta_i|$$

Diamond constraint. Drives coefficients to **exactly zero** — automatic feature selection.

L2 (RIDGE)

$$\Omega = \sum_{i=1}^{p} \beta_i^2$$

Circle constraint. Shrinks coefficients **toward zero** but never exactly zero.

ELASTIC NET

$$\Omega = \alpha \sum |\beta_i| + (1 - \alpha) \sum \beta_i^2$$

Combines L1 and L2. α mixes sparsity with grouping.

ADVANCED

BAYESIAN INTERPRETATION OF L2

L2 regularisation is equivalent to placing a **Gaussian prior** on the coefficients:

$$\beta_i \sim \mathcal{N}(0, \tau^2) \quad \Rightarrow \quad \text{MAP estimate} = \text{Ridge solution}$$

where $\tau^2 = \sigma^2 / \lambda$. The prior says "I believe coefficients are small."

L1 = LAPLACE PRIOR

The Laplace distribution has a sharp peak at zero, which is why L1 produces exact zeros (sparsity).

L2 = GAUSSIAN PRIOR

The Gaussian distribution is smooth and round, which is why L2 shrinks uniformly but never reaches zero.

Connection to M2: This bridges Bayesian thinking (M2.1) with regularisation. Regularisation is not a hack — it is a principled statement of prior belief about the world.

CROSS-VALIDATION

Problem: A single train/test split wastes data and gives noisy estimates. **Solution:** Rotate the test fold.

Method	How It Works	When to Use
k-Fold	Split into k folds; each fold is test once	Default for tabular data (k=5 or 10)
Stratified k-Fold	Preserve class proportions in each fold	Imbalanced classification
Time-Series Split	Walk-forward: train on past, test on future	Temporal data (never use future to predict past)
GroupKFold	Keep grouped observations together	Same patient, same company, repeated measures
Nested CV	Outer: model selection. Inner: hyperparameters	Unbiased model comparison (Lesson 3.7)

KAILASH BRIDGE: TRAININGPIPELINE

TRAININGPIPELINE

Unified training interface with built-in cross-validation, regularisation, and metric tracking.

```python
from kailash_ml import TrainingPipeline

pipeline = TrainingPipeline(
    model="ridge_regression",
    cv_strategy="stratified_kfold",
    cv_folds=5,
    regularisation={"type": "l2", "lambda": 0.1}
)
results = pipeline.fit(X_train, y_train)
print(results.cv_scores)  # per-fold and mean scores
```

Why TrainingPipeline? It enforces proper CV discipline: the scaler is fitted inside each fold, not on the full dataset. This prevents preprocessing leakage automatically.

LESSON 3.2 SUMMARY

- **Bias-variance decomposition:** EPE = Bias² + Variance + Noise
- **L1 (Lasso):** drives coefficients to zero — feature selection
- **L2 (Ridge):** shrinks coefficients — Gaussian prior
- **Elastic Net:** combines L1 + L2 with mixing parameter α
- **Cross-validation:** match CV strategy to data structure
- **Engine:** `TrainingPipeline` with built-in CV

Exercise 3.2: Demonstrate bias-variance tradeoff by varying polynomial degree on HDB data. Compare L1 vs L2 vs ElasticNet. Implement nested CV for unbiased model selection.

LESSON 3.3

THE COMPLETE SUPERVISED MODEL ZOO

"There is no single best model. The right model depends on your data."

NO FREE LUNCH THEOREM

There is no single model that is best for all problems.

> **Wolpert & Macready (1997):** Averaged over ALL possible data distributions, every learning algorithm performs equally well. Any advantage on one class of problems is paid for by disadvantage on another.

What this means in practice:

- You MUST try multiple models on YOUR data
- "Always use XGBoost" is wrong (even though it wins many Kaggle competitions)
- The best model depends on: data size, feature types, noise level, interpretability needs

> **Kailash bridge:** `AutoMLEngine` systematically compares multiple models so you do not have to guess.

THEORY

SUPPORT VECTOR MACHINES (SVM)

Idea: Find the hyperplane that maximises the margin between classes.

$$\text{Maximise } \frac{2}{\|\mathbf{w}\|} \quad \text{subject to } y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \forall i$$

w = weight vector (normal to hyperplane), b = bias, y_i ∈ {-1, +1}

KEY CONCEPTS

- **Support vectors:** Points closest to the boundary (they define it)
- **Soft margin (C):** Allow some misclassifications; C controls tolerance
- **Kernel trick:** Map to higher dimensions without computing them

WHEN TO USE

- High-dimensional data (text, genomics)
- Clear margin of separation
- Small-to-medium datasets
- Kernels: linear, RBF, polynomial

K-NEAREST NEIGHBORS (KNN)

Idea: Classify a new point by the majority vote of its k nearest neighbors.

HOW IT WORKS

- Store all training data (no training step)
- For a new point: find k closest neighbors
- Classification: majority vote
- Regression: average of k neighbors

KEY DECISIONS

- **k:** Small k = complex boundary (overfit). Large k = smooth boundary (underfit).
- **Distance:** Euclidean, Manhattan, cosine
- **Scaling:** Features MUST be scaled (otherwise distance is dominated by large-range features)

Curse of dimensionality: In high dimensions, all points become equidistant. KNN degrades above ~20 features. Use feature selection (Lesson 3.1) first.

NAIVE BAYES

Idea: Apply Bayes' theorem with the "naive" assumption that features are independent.

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y) \cdot P(y)}{P(\mathbf{x})} \propto P(y) \prod_{i=1}^{p} P(x_i \mid y)$$

The naive independence assumption: P(x_1, x_2, ..., x_p | y) = P(x_1|y) P(x_2|y) ... P(x_p|y)

VARIANTS

- **GaussianNB:** Continuous features (assumes normal)
- **MultinomialNB:** Count features (text / bag-of-words)
- **BernoulliNB:** Binary features (word presence/absence)

WHEN TO USE

- Text classification (spam, sentiment)
- Fast baseline model
- Very small training data
- Surprisingly effective despite "wrong" assumption

DECISION TREES

Idea: Recursively split data on feature thresholds to create a tree of decisions.

GINI IMPURITY

$$G = 1 - \sum_{i=1}^{C} p_i^2$$

p_i = proportion of class i in the node. G = 0 means pure node.

INFORMATION GAIN (ENTROPY)

$$\text{IG} = H(\text{parent}) - \sum_{j} w_j \cdot H(\text{child}_j)$$

H = -∑ p_i log_2(p_i). Choose the split with highest IG.

Pruning: Pre-pruning (max_depth, min_samples_leaf) or post-pruning (cost-complexity).

When to use: Highly interpretable. Handles mixed feature types. Non-linear boundaries. But prone to overfitting without pruning.

RANDOM FORESTS

Idea: Combine many decorrelated trees to reduce variance while keeping low bias.

BAGGING (BOOTSTRAP AGGREGATING)

- Each tree trains on a bootstrap sample (random draw with replacement)
- Each split considers a random subset of features
- Final prediction: majority vote (classification) or average (regression)

KEY PROPERTIES

- **OOB estimation:** ~36.8% of samples not in each bootstrap → free validation set
- **Feature importance:** Mean decrease in impurity or permutation importance
- **Robust default:** Hard to overtune, handles missing data

Bias-variance connection: Bagging reduces variance (averaging decorrelated predictions). Individual trees have low bias but high variance. The forest keeps the low bias and fixes the variance problem.

MODEL COMPARISON FRAMEWORK

Model	Accuracy	Speed	Interpretability	Best For
Linear/Ridge	Moderate	Fast	High	Baseline, explanatory
SVM	High	Slow (large N)	Low (kernel)	High-dim, clear margin
KNN	Moderate	Slow (predict)	Moderate	Small data, local patterns
Naive Bayes	Moderate	Very fast	High	Text, fast baseline
Decision Tree	Low-Moderate	Fast	Very high	Explanations, rules
Random Forest	High	Moderate	Moderate	Robust default

Rule of thumb: Start with Random Forest as your baseline. If interpretability matters, use a tree or linear model. If accuracy is paramount, move to gradient boosting (Lesson 3.4).

KAILASH BRIDGE: AUTOMLENGINE

AUTOMLENGINE

Systematically trains and compares multiple models with consistent evaluation.

```python
from kailash_ml import AutoMLEngine

auto = AutoMLEngine(
    task="classification",
    models=["random_forest", "svm", "knn", "naive_bayes", "decision_tree"],
    cv_folds=5,
    metric="f1_weighted"
)
leaderboard = auto.fit(X_train, y_train)
print(leaderboard.ranking())  # sorted by metric
```

Why AutoMLEngine? It enforces consistent evaluation: same CV splits, same preprocessing, same metrics. Human comparison across Jupyter cells often introduces subtle inconsistencies.

LESSON 3.3 SUMMARY

- **No Free Lunch:** No single best model — match algorithm to problem
- **SVM:** Margin maximisation + kernel trick for non-linear boundaries
- **KNN:** Instance-based; simple but curse of dimensionality
- **Naive Bayes:** Bayesian with independence assumption; fast baseline
- **Decision Trees:** Gini / entropy splits; interpretable but overfit
- **Random Forest:** Bagging + feature subsampling; robust default
- **Engine:** `AutoMLEngine` for systematic comparison

> **Exercise 3.3:** Train all 5 model families on e-commerce customer classification. Compare accuracy, F1, training time. Produce model comparison table.

LESSON 3.4

GRADIENT BOOSTING DEEP DIVE

"The algorithm that wins most tabular ML competitions — and powers most production ML systems."

BAGGING VS BOOSTING

BAGGING (RANDOM FOREST)

- Train trees **in parallel**
- Each tree is independent
- Average predictions
- Reduces **variance**

BOOSTING (XGBOOST, LIGHTGBM)

- Train trees **sequentially**
- Each tree corrects the previous tree's errors
- Sum weighted predictions
- Reduces **bias**

Bias-variance lens: Random Forest starts with low-bias, high-variance trees and averages to reduce variance. Boosting starts with high-bias, low-variance stumps and adds corrections to reduce bias. Both achieve low total error — from opposite directions.

ADABOOST — CONCEPTUAL WARMUP

Adaptive Boosting: Reweight misclassified samples so the next tree focuses on what was hard.

1. Train a weak learner (decision stump)
2. Find misclassified samples
3. **Increase weights** on misclassified samples
4. Train next weak learner on reweighted data
5. Repeat. Final prediction = weighted vote of all weak learners.

> **Intuition:** Each new tree specialises in the cases the previous trees got wrong. The ensemble gradually handles all the hard cases.

AdaBoost is conceptually clean but has been superseded by gradient boosting, which is more general and handles diverse loss functions.

XGBOOST — THE MATH

XGBoost uses a **2nd-order Taylor expansion** of the loss function to find optimal splits efficiently.

OBJECTIVE AT ROUND T

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} L(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t)$$

TAYLOR EXPAND AROUND THE CURRENT PREDICTION

$$\approx \sum_{i=1}^{n} \left[g_i \cdot f_t(\mathbf{x}_i) + \tfrac{1}{2} h_i \cdot f_t^2(\mathbf{x}_i) \right] + \Omega(f_t)$$

g_i = first derivative (gradient), h_i = second derivative (Hessian) of loss w.r.t. prediction

Why 2nd order? The Hessian gives curvature information, enabling Newton-like updates. This converges faster than gradient-only methods and naturally adapts the learning rate per sample.

XGBOOST SPLIT GAIN FORMULA

For each potential split, XGBoost computes the gain — how much the objective improves:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

G_L, G_R = sum of gradients in left/right child. H_L, H_R = sum of Hessians. λ = L2 regularisation on leaf weights. γ = min split loss.

Parameter	Role	Effect
λ	L2 regularisation on leaf weights	Larger λ = smoother, less overfitting
γ	Minimum split loss	Larger γ = fewer splits, simpler tree
η (learning rate)	Shrink each tree's contribution	Smaller η = more trees needed but better generalisation

LIGHTGBM AND CATBOOST

LIGHTGBM

- **GOSS:** Keep top-a% gradient samples, randomly sample b% of small gradients
- **Histogram-based:** Bin continuous features into 256 bins for faster splits
- **Leaf-wise growth:** Grow the leaf with highest loss reduction (vs level-wise)
- Best for: large datasets, speed-critical

CATBOOST

- **Ordered boosting:** Prevents target leakage during training
- **Native categorical:** Handles categories without manual encoding
- **Symmetric trees:** Same split at every node of a level
- Best for: datasets with many categorical features

GOSS formula: Keep all instances with gradients in the top a%, randomly sample b% from the rest. Weight the sampled instances by (1-a)/b to maintain the gradient distribution.

BOOSTING FAMILY COMPARISON

Aspect	XGBoost	LightGBM	CatBoost
Tree growth	Level-wise	Leaf-wise	Symmetric
Speed	Moderate	Fastest	Moderate
Categoricals	Manual encoding	Manual encoding	Native
Overfitting	λ, γ, depth	λ, γ, leaves	Ordered boosting
Best for	General tabular	Large data, speed	Many categoricals

Practical advice: Start with LightGBM for speed. Switch to CatBoost if you have many categorical features. XGBoost is the safe default with the most community support.

KAILASH BRIDGE: GRADIENT BOOSTING

TRAININGPIPELINE — BOOSTING MODELS

```python
from kailash_ml import TrainingPipeline

# Compare all three boosting frameworks
for model_name in ["xgboost", "lightgbm", "catboost"]:
    pipeline = TrainingPipeline(
        model=model_name,
        cv_strategy="stratified_kfold",
        cv_folds=5,
        metric="f1_weighted",
        hyperparams={"n_estimators": 500, "learning_rate": 0.05}
    )
    results = pipeline.fit(X_train, y_train)
    print(f"{model_name}: {results.mean_score:.4f}")
```

LESSON 3.4 SUMMARY

- **Bagging vs Boosting:** Variance reduction vs bias reduction
- **XGBoost:** 2nd-order Taylor expansion, split gain with λ and γ
- **LightGBM:** GOSS + histogram + leaf-wise growth = speed
- **CatBoost:** Ordered boosting + native categorical support
- **In practice:** Try all three; differences are marginal on most datasets

Exercise 3.4: Train XGBoost, LightGBM, CatBoost on credit scoring data. Compare accuracy, training time, feature importance. Tune key hyperparameters. Justify selection.

LESSON 3.5

MODEL EVALUATION, IMBALANCE, AND CALIBRATION

"A model is only as trustworthy as the metric you use to judge it."

REMEMBER THE AML MODEL?

99.9% accuracy

But the compliance team called it useless.

10 million transactions. Only ~100 suspicious. The model said "everything is fine" and was 99.99% accurate.

> **When only 0.001% of data is positive, accuracy is the WRONG metric.** The model needs to find those 100 needles in 10 million haystacks. That requires precision and recall.

This lesson gives you the tools to evaluate models *honestly*.

CLASSIFICATION METRICS

$$\text{Precision} = \frac{TP}{TP + FP}$$

"Of the alerts we raised, how many were real?"

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Harmonic mean — punishes imbalance between P and R

$$\text{Recall} = \frac{TP}{TP + FN}$$

"Of all real cases, how many did we catch?"

$$\text{AUC} = \int_0^1 \text{TPR}(t)\, d(\text{FPR}(t))$$

Area under ROC curve: probability a random positive ranks above a random negative

THEORY

REGRESSION METRICS

Metric	Formula	When to Use
R^2	$1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$	Overall fit quality (default)
Adj. R^2	$1 - (1 - R^2)\frac{n-1}{n-p-1}$	Penalises adding useless features
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$	Robust to outliers
RMSE	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$	Same units as target; penalises large errors
MAPE	$\frac{100}{n}\sum\left\lvert\frac{y_i - \hat{y}_i}{y_i}\right\rvert$	Percentage error; business-friendly

CHOOSING THE RIGHT METRIC

Situation	Metric	Why
Balanced classification	Accuracy, F1	All classes matter equally
Imbalanced classification	Precision-Recall AUC, F1	Accuracy hides poor minority performance
Cost-sensitive (medical)	Recall (sensitivity)	Missing a disease is worse than false alarm
Cost-sensitive (spam)	Precision	False positives lose real emails
Probabilistic output	Log loss, Brier score	Evaluate predicted probabilities, not just labels
Regression (general)	RMSE	Penalises large errors
Regression (business)	MAPE	Stakeholders understand percentages

HANDLING CLASS IMBALANCE

Problem: When one class dominates (99:1), models learn to predict the majority class and ignore the minority.

SAMPLING APPROACHES

- **SMOTE:** Synthesise new minority samples by interpolating between existing ones
- **Limitations:** Fails on high-dimensional data; generates unrealistic samples at class boundaries

ALGORITHMIC APPROACHES

- **Class weights:** Weight minority class higher in the loss function
- **Threshold moving:** Adjust the classification threshold (not just 0.5)
- **Focal loss:** Down-weight easy examples automatically

Do not just SMOTE and hope. Class weights and focal loss are usually more effective and do not generate synthetic data that may not reflect reality.

FOCAL LOSS

Focal loss automatically down-weights easy examples so the model focuses on hard ones:

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

α_t = class balancing weight. γ = focusing parameter (typically 2). p_t = predicted probability of the true class.

HOW Γ WORKS

- γ = 0: standard cross-entropy
- γ = 2: well-classified samples (p_t > 0.9) get ~100x less loss
- Effect: model stops wasting capacity on easy majority examples

WHEN TO USE

- Extreme imbalance (fraud, rare diseases)
- When the model is overconfident on easy examples
- Originally from object detection (Lin et al. 2017)

PROBABILITY CALIBRATION

Problem: Most models output scores, not true probabilities. A model might say "80% chance of default" when the true rate is 50%.

PLATT SCALING

Fit a logistic regression on model outputs to calibrate probabilities. Parametric: assumes sigmoid shape.

ISOTONIC REGRESSION

Fit a non-decreasing step function. Non-parametric: more flexible but needs more data.

$$\text{Brier Score} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2$$

Measures calibration quality. Lower is better. 0 = perfect calibration.

Why calibrate? When decisions depend on predicted probabilities (lending thresholds, medical triage, insurance pricing), miscalibrated probabilities lead to wrong decisions.

LESSON 3.5 SUMMARY

- **Classification:** Precision, Recall, F1, AUC — match metric to business cost
- **Regression:** R^2, RMSE, MAE, MAPE — use RMSE as default
- **Imbalance:** Class weights and focal loss over SMOTE
- **Focal loss:** $-\alpha_t(1-p_t)^\gamma \log(p_t)$
- **Calibration:** Platt scaling or isotonic regression; Brier score to measure

Exercise 3.5: Train on imbalanced credit scoring data. Compare accuracy vs F1 vs AUC. Apply SMOTE and cost-sensitive approaches. Calibrate with Platt scaling. Show improvement on calibration plot.

LESSON 3.6

INTERPRETABILITY AND FAIRNESS

"If you cannot explain why the model made a decision, should you trust the decision?"

WHY INTERPRETABILITY MATTERS

REGULATORY REQUIREMENTS

- EU AI Act: "right to explanation" for high-risk AI
- MAS FEAT: Fairness, Ethics, Accountability, Transparency (Singapore)
- ECOA (US): Cannot deny credit without explanation

PRACTICAL BENEFITS

- Debug models: find leakage, spurious correlations
- Build trust: stakeholders accept what they understand
- Detect bias: systematic discrimination in predictions
- Improve models: understand what the model learned

SHAP — SHAPLEY ADDITIVE EXPLANATIONS

Based on cooperative game theory: how much does each feature contribute to the prediction?

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[f(S \cup \{i\}) - f(S) \right]$$

φ_i = Shapley value for feature i. S = subset of features. F = all features. f(S) = model prediction using only features in S.

SHAPLEY AXIOMS

Efficiency: Contributions sum to the total prediction minus baseline

Symmetry: Equal contributors get equal credit

Dummy: Irrelevant features get zero credit

Linearity: Additive across combined games

SHAP VARIANTS AND PLOTS

COMPUTATION METHODS

- **TreeSHAP:** Exact and fast for tree models ($O(TLD^2)$ per prediction)
- **KernelSHAP:** Model-agnostic but slower (weighted linear regression on coalitions)
- **DeepSHAP:** Efficient for deep learning (backpropagation-based)

PLOT TYPES

- **Summary plot:** Global feature importance + direction of effect
- **Waterfall:** Single prediction decomposed into feature contributions
- **Dependence:** How one feature's SHAP value varies with its value
- **Force:** Compact single-prediction explanation

THEORY

LIME — LOCAL INTERPRETABLE EXPLANATIONS

Idea: Perturb the input, observe how predictions change, fit a local linear model.

1. Take the instance to explain
2. Generate perturbed samples nearby
3. Get model predictions on the perturbed samples
4. Fit a **simple linear model** weighted by proximity
5. The linear model's coefficients explain which features matter *locally*

> **LIME vs SHAP:** LIME is faster but less theoretically grounded (no uniqueness guarantees). SHAP satisfies the Shapley axioms. In practice, use SHAP for rigorous analysis and LIME for quick local explanations.

FAIRNESS METRICS

Metric	Definition	Threshold
Disparate Impact	$\frac{P(\hat{Y}=1 \mid G=\text{minority})}{P(\hat{Y}=1 \mid G=\text{majority})}$	Should be > 0.8 (80% rule)
Equalized Odds	TPR and FPR equal across groups	Difference < 0.05
Calibration Parity	Predicted probabilities equally reliable across groups	Brier score difference < 0.02

ADVANCED

THE FAIRNESS IMPOSSIBILITY THEOREM

Chouldechova (2017) / Kleinberg et al. (2016): A classifier CANNOT simultaneously satisfy demographic parity, equalized odds, AND calibration — except in trivial cases (perfect prediction or equal base rates).

What this means in practice:

- You must **choose** which fairness criterion matters most for your context
- Lending: calibration parity (predicted default rates must be accurate per group)
- Hiring: disparate impact (selection rates must be comparable)
- Criminal justice: equalized odds (error rates must be balanced)

Implication: Fairness is an engineering and ethical decision, not a mathematical one. There is no "fair" algorithm — only algorithms whose fairness tradeoffs are explicit and documented.

ADVANCED

ALE — ACCUMULATED LOCAL EFFECTS

Problem with Partial Dependence Plots (PDP): When features are correlated, PDP creates unrealistic data points.

ALE solution: Only consider how predictions change within small windows of the feature, avoiding extrapolation to unrealistic feature combinations.

> **When to prefer ALE over PDP:** Always, when features are correlated (which is almost always). ALE is unbiased; PDP can show spurious effects from impossible feature combinations. See Apley & Zhu (2020).

LESSON 3.6 SUMMARY

- **SHAP:** Game-theoretic attribution with 4 axioms and Shapley value formula
- **LIME:** Local linear approximation — fast but less rigorous
- **ALE:** Unbiased alternative to partial dependence for correlated features
- **Fairness:** Disparate impact, equalized odds, calibration parity
- **Impossibility:** Cannot satisfy all fairness criteria simultaneously

Exercise 3.6: Compute SHAP values for the credit scoring model. Generate summary and waterfall plots. Measure disparate impact across demographic groups. Document findings.

LESSON 3.7

WORKFLOW ORCHESTRATION, MODEL REGISTRY, AND HYPERPARAMETER SEARCH

"Good data science is reproducible. Great data science is automated."

WORKFLOWBUILDER — NODE-BASED PIPELINES

WORKFLOWBUILDER

Build ML pipelines as directed acyclic graphs (DAGs) of nodes.

```
from kailash import WorkflowBuilder, Runtime

wf = WorkflowBuilder("ml-pipeline")
wf.add_node("load", data_loader_node)
wf.add_node("preprocess", preprocessing_node)
wf.add_node("train", training_node)
wf.add_node("evaluate", evaluation_node)
wf.connect("load", "preprocess")
wf.connect("preprocess", "train")
wf.connect("train", "evaluate")

runtime = Runtime()
results = runtime.execute(wf.build())
```

THEORY

CUSTOM NODES

Every workflow step is a node. Kailash provides built-in nodes and lets you create custom ones.

CUSTOM NODE EXAMPLE

```python
from kailash import Node, register_node

@register_node("feature_engineer")
class FeatureEngineerNode(Node):
    def execute(self, inputs):
        df = inputs["data"]
        df = df.with_columns(
            (pl.col("floor_area") * pl.col("storey")).alias("size_score"),
            pl.col("month").dt.month().alias("sale_month"),
        )
        return {"data": df, "feature_count": len(df.columns)}
```

Also available: `PythonCodeNode` (inline code), `ConditionalNode` (branching logic).

THEORY

BAYESIAN HYPERPARAMETER SEARCH

HYPERPARAMETERSEARCH

```
from kailash_ml import HyperparameterSearch, SearchSpace, SearchConfig

space = SearchSpace({
    "n_estimators": (100, 1000),
    "max_depth": (3, 12),
    "learning_rate": (0.01, 0.3, "log"),
    "subsample": (0.6, 1.0),
})

search = HyperparameterSearch(
    model="xgboost",
    search_space=space,
    config=SearchConfig(strategy="bayesian", n_trials=50, cv_folds=5),
    metric="f1_weighted"
)
best = search.run(X_train, y_train)
print(best.params, best.score)
```

Why Bayesian? Grid search is exhaustive (too slow). Random search is better but uninformed. Bayesian optimisation builds a surrogate model of the objective and samples where improvement is most likely.

MODEL REGISTRY — VERSIONING AND LIFECYCLE

MODELREGISTRY

```
from kailash_ml import ModelRegistry, ModelSignature, MetricSpec

registry = ModelRegistry()
registry.register(
    model=best_model,
    name="credit-scorer-v2",
    signature=ModelSignature(
        inputs={"age": "float", "income": "float", "credit_history": "int"},
        output={"default_probability": "float"}
    ),
    metrics={"f1": 0.87, "auc": 0.93, "brier": 0.12},
    stage="staging"
)
```

Experiment → Staging → Production → Retired

MODEL LIFECYCLE

Experiment → Register → Stage → Validate → Promote → Serve → Monitor → Retire

Stage	Gate	Who Decides
Experiment → Stage	Metrics meet minimum thresholds	Automated (pipeline)
Stage → Production	A/B test on shadow traffic passes	Human (model owner)
Production → Retired	Drift detected or new model promoted	Automated + human review

ADVANCED

NESTED CROSS-VALIDATION — UNBIASED MODEL SELECTION

Problem: If you use the same CV to both tune hyperparameters AND estimate performance, you are overfitting to the validation set.

Solution — Nested CV:

- **Outer loop** (5 folds): estimates true generalisation performance
- **Inner loop** (5 folds): tunes hyperparameters within each outer fold
- Result: unbiased estimate of how well a tuned model generalises

Expensive (25 total fits for 5×5) but the gold standard for model comparison.

Kailash bridge: `TrainingPipeline` supports nested CV natively: `cv_strategy="nested"` with separate inner and outer fold counts.

LESSON 3.7 SUMMARY

- **WorkflowBuilder:** DAG-based pipeline orchestration with custom nodes
- **HyperparameterSearch:** Bayesian optimisation > grid search > random search
- **ModelRegistry:** Version, sign, and lifecycle-manage models
- **Nested CV:** Unbiased model selection (outer) + tuning (inner)
- **Model lifecycle:** Experiment → Stage → Production → Retire

Exercise 3.7: Build automated ML workflow: data loading → preprocessing → training → evaluation → conditional promotion. Register best model with signature.

LESSON 3.8

PRODUCTION PIPELINE — DATAFLOW, DRIFT, AND DEPLOYMENT

"A model that works on your laptop and nowhere else is a science project, not a product."

DATAFLOW — DATABASE PERSISTENCE

DATAFLOW

Zero-config database operations for persisting ML results.

```
from kailash_dataflow import DataFlow, field

db = DataFlow()

@db.model
class PredictionResult:
    customer_id: str = field(primary_key=True)
    score: float = field()
    model_version: str = field()
    predicted_at: str = field()

# CRUD operations
await db.express.create(PredictionResult,
    customer_id="C001", score=0.73,
    model_version="v2.1", predicted_at="2024-03-15"
)
results = await db.express.list(PredictionResult)
```

ASYNC/AWAIT — QUICK PRIMER

DataFlow uses `async/await` for database operations. Here is all you need to know:

```
# Synchronous (blocking)
result = db.query("SELECT ...")
# Python waits. Nothing else runs.

# Asynchronous (non-blocking)
result = await db.query("SELECT ...")
# Python can do other work while waiting.
```

WHY ASYNC?

- Database I/O is slow (network round-trip)
- Async lets the program do other things while waiting
- Critical for production servers handling many requests

Rule of thumb: If you see `await`, the function must be `async def`. That is the only syntax difference.

DATAFLOW — CONNECTIONMANAGER

CONNECTION LIFECYCLE

```python
from kailash_dataflow import DataFlow, ConnectionManager

db = DataFlow()

# Context manager ensures clean connection lifecycle
async with ConnectionManager(db) as conn:
    await conn.create_tables()
    await db.express.create(PredictionResult, ...)
    results = await db.express.list(PredictionResult)
# Connection automatically closed on exit
```

Always use ConnectionManager as a context manager. Forgetting to close connections causes resource leaks in production. The `async with` pattern guarantees cleanup.

DRIFTMONITOR — DETECTING MODEL DECAY

Problem: Models degrade over time because the world changes. A model trained on 2023 data may be wrong by 2025.

TYPES OF DRIFT

- **Data drift:** Input feature distributions change
- **Concept drift:** The relationship between features and target changes
- **Performance drift:** Model accuracy degrades over time

DETECTION METHODS

- **PSI:** Population Stability Index — compares feature distributions
- **KS test:** Kolmogorov-Smirnov — compares CDFs
- **Performance tracking:** Monitor metrics on labelled data

PSI AND KS TEST

PSI (POPULATION STABILITY INDEX)

$$\text{PSI} = \sum_{i=1}^{B} (p_i^{\text{new}} - p_i^{\text{ref}}) \cdot \ln\left(\frac{p_i^{\text{new}}}{p_i^{\text{ref}}}\right)$$

- PSI < 0.1: no significant change
- 0.1 ≤ PSI < 0.25: moderate change
- PSI ≥ 0.25: significant drift

KS TEST (KOLMOGOROV-SMIRNOV)

$$D = \sup_x |F_{\text{ref}}(x) - F_{\text{new}}(x)|$$

- Measures max distance between CDFs
- p-value indicates significance
- Distribution-free (no assumptions)

KAILASH BRIDGE: DRIFTMONITOR

DRIFTMONITOR

```python
from kailash_ml import DriftMonitor, DriftSpec

monitor = DriftMonitor(
    reference_data=X_train,
    spec=DriftSpec(
        features=["age", "income", "credit_history"],
        methods=["psi", "ks"],
        thresholds={"psi": 0.25, "ks_pvalue": 0.01},
        frequency="daily"
    )
)

# Check new production data for drift
report = monitor.check(X_production)
if report.has_drift:
    print(f"Drift detected: {report.drifted_features}")
    # Trigger retraining pipeline
```

MODEL CARDS — DOCUMENTING YOUR MODEL

A model card (Mitchell et al. 2019) documents everything a stakeholder needs to know:

MODEL DETAILS

Name, version, type, training date, owner

PERFORMANCE

Metrics by subgroup, calibration, confidence intervals

INTENDED USE

Primary use cases, out-of-scope uses

FAIRNESS ANALYSIS

Disparate impact, equalized odds, known biases

TRAINING DATA

Dataset description, preprocessing, limitations

LIMITATIONS & RISKS

Known failure modes, drift sensitivity, ethical concerns

ADVANCED

CONFORMAL PREDICTION — UNCERTAINTY QUANTIFICATION

Problem: Most models give point predictions. Decision-makers need to know how uncertain the prediction is.

> **Conformal prediction (Vovk et al. 2005):** Distribution-free prediction intervals with guaranteed coverage. If you request 90% coverage, the true value falls within the interval at least 90% of the time — regardless of the underlying distribution.

How it works:

1. Train model on training set
2. Compute nonconformity scores on calibration set
3. For new prediction: interval = prediction ± quantile of calibration scores

No distributional assumptions. Works with any model. Connects to calibration (Lesson 3.5).

THE FULL PRODUCTION PIPELINE

Train → Evaluate → Calibrate → Register → Persist to DataFlow → Promote → Serve → Monitor for Drift → Model Card

ENGINE MAP FOR EACH STAGE

Stage	Engine
Train + Evaluate	`TrainingPipeline`, `AutoMLEngine`
Tune	`HyperparameterSearch`
Register + Promote	`ModelRegistry`
Persist	`DataFlow`
Monitor	`DriftMonitor`
Orchestrate	`WorkflowBuilder`

MLOPS — CI/CD FOR MACHINE LEARNING

TRADITIONAL CI/CD

- Code changes trigger builds
- Tests validate correctness
- Deploy if tests pass

ML CI/CD (MLOPS)

- Code OR data changes trigger retraining
- Tests validate correctness AND performance
- Deploy if metrics pass AND no drift AND fairness checks pass

Key difference: In traditional software, code changes are the trigger. In ML, data changes can require redeployment even if the code is unchanged. This is why drift monitoring (DriftMonitor) is essential.

LESSON 3.8 SUMMARY

- **DataFlow:** Persist predictions with schema validation and connection management
- **DriftMonitor:** PSI + KS test for feature drift detection
- **Model cards:** Document performance, fairness, and limitations
- **Conformal prediction:** Distribution-free uncertainty intervals
- **MLOps:** CI/CD for ML — code AND data trigger deployment

Exercise 3.8: Deploy credit scoring model as full production pipeline. Persist to DataFlow. Set up DriftMonitor. Generate model card. Simulate drift and verify detection.

THEORY

MODULE 3 — KEY FORMULAS REFERENCE

Concept	Formula
Bias-Variance	$\text{EPE} = \text{Bias}^2 + \text{Var} + \sigma^2$
L1 Penalty	$\lambda \sum \lvert\beta_i\rvert$
L2 Penalty	$\lambda \sum \beta_i^2$
Gini Impurity	$G = 1 - \sum p_i^2$
XGBoost Gain	$\frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$
Precision	$\frac{TP}{TP+FP}$
Recall	$\frac{TP}{TP+FN}$
F1 Score	$\frac{2 \cdot P \cdot R}{P+R}$
Focal Loss	$-\alpha_t (1-p_t)^\gamma \log(p_t)$
Shapley Value	$\phi_i = \sum_S \frac{\lvert S\rvert!(\lvert F\rvert - \lvert S\rvert - 1)!}{\lvert F\rvert!}[f(S \cup \{i\}) - f(S)]$
Brier Score	$\frac{1}{N}\sum (p_i - y_i)^2$
PSI	$\sum (p_i^{new} - p_i^{ref}) \ln(p_i^{new} / p_i^{ref})$

MODULE 3 — COMPLETE ENGINE MAP

Engine	Lesson	Purpose
`FeatureEngineer`	3.1	Generate and select features
`FeatureStore`	3.1	Versioned features with point-in-time correctness
`TrainingPipeline`	3.2-3.4	Unified training with CV and regularisation
`AutoMLEngine`	3.3	Systematic model comparison
`EnsembleEngine`	3.5	Stacking and blending (preview for M4)
`HyperparameterSearch`	3.7	Bayesian hyperparameter optimisation
`ModelRegistry`	3.7	Model versioning and lifecycle
`WorkflowBuilder`	3.7	DAG-based pipeline orchestration
`DataFlow`	3.8	Database persistence for ML results
`DriftMonitor`	3.8	Feature and performance drift detection

MODULE 3 — WHAT YOU CAN NOW DO

SCIENCE

- Engineer features from domain knowledge
- Derive bias-variance decomposition
- Apply L1, L2, ElasticNet regularisation
- Train SVM, KNN, NB, trees, forests, boosting
- Evaluate with precision, recall, F1, AUC
- Handle imbalance with focal loss
- Interpret with SHAP and LIME
- Measure fairness quantitatively

ENGINEERING

- Build DAG-based ML workflows
- Run Bayesian hyperparameter search
- Register and lifecycle-manage models
- Persist results with DataFlow
- Monitor for drift with PSI and KS
- Document with model cards
- Calibrate probabilities
- Quantify uncertainty with conformal prediction

MODULE 3 — ASSESSMENT

QUIZ (40%)

- Bias-variance decomposition
- Regularisation (L1 vs L2 behaviour)
- Model selection rationale
- Metric selection for given scenarios
- XGBoost split gain formula
- Fairness impossibility theorem

ML PIPELINE PROJECT (60%)

- Full pipeline from raw data to monitored model
- Feature engineering with domain rationale
- Model comparison (minimum 3 algorithms)
- Proper evaluation with appropriate metrics
- SHAP-based interpretability
- DataFlow persistence + DriftMonitor setup
- Model card

EXERCISE SUMMARY

Exercise	Dataset	Key Skills
3.1	HDB resale prices	Feature engineering, 3 selection methods
3.2	HDB resale prices	Bias-variance demo, regularisation comparison
3.3	E-commerce customers	5 model families, consistent comparison
3.4	Credit scoring	XGBoost vs LightGBM vs CatBoost
3.5	Credit scoring (imbalanced)	Metrics, SMOTE vs class weights, calibration
3.6	Credit scoring	SHAP, LIME, fairness metrics
3.7	Credit scoring	WorkflowBuilder, hyperparameter search, registry
3.8	Credit scoring	DataFlow, DriftMonitor, model card

COMMON MISTAKES TO AVOID

Mistake	Why It Fails	What to Do Instead
Using accuracy on imbalanced data	Hides poor minority-class performance	Use F1, PR-AUC, or custom cost matrix
Fitting scaler before splitting	Preprocessing leakage	Fit inside CV loop (TrainingPipeline does this)
Using future data for temporal features	Temporal leakage	Time-travel test every feature
Only trying one model	No Free Lunch — one model is not best	AutoMLEngine to compare 3+ models
Tuning then reporting same CV score	Optimistic performance estimate	Nested CV for unbiased estimate
Deploying without drift monitoring	Model silently degrades	DriftMonitor with alerting thresholds

MODEL SELECTION DECISION TREE

Is interpretability required?

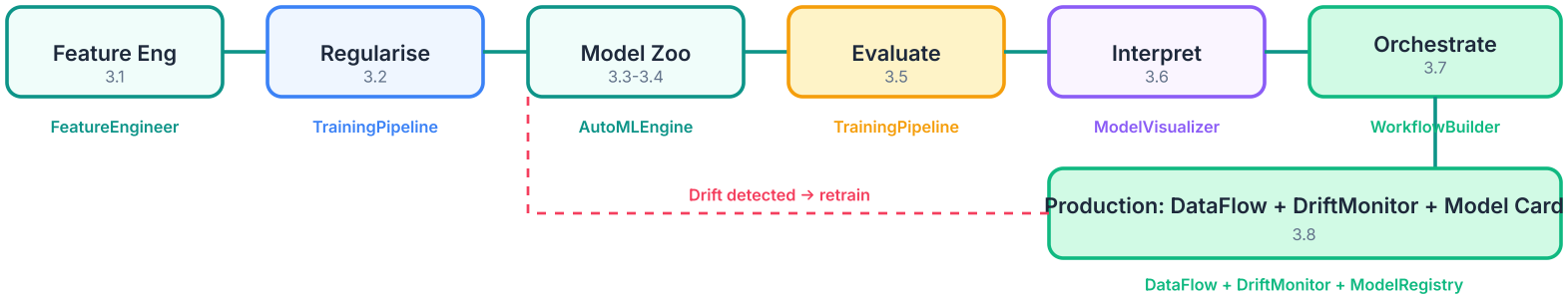
- **Yes →** Linear model (with regularisation) or decision tree
- **No →** Is dataset large (>10K rows)?
  - **Yes →** Gradient boosting (LightGBM for speed, CatBoost for categoricals)
  - **No →** Random Forest or SVM (with kernel)

Is data primarily text? → Start with Naive Bayes, then boosting

Is there extreme imbalance? → Use focal loss + class weights with any model

Always: Compare 3+ models with AutoMLEngine. Evaluate with the metric that matches business cost.

THE COMPLETE SUPERVISED PIPELINE



COMING UP: MODULE 4

MODULE 4 — UNSUPERVISED ML, ANOMALY DETECTION & NLP

- **Clustering:** K-means, DBSCAN, hierarchical
- **Dimensionality reduction:** PCA, t-SNE, UMAP
- **Anomaly detection:** Isolation Forest, autoencoders
- **NLP:** Text preprocessing, embeddings, topic modelling
- **New engines:** EnsembleEngine, InferenceServer

WHAT CARRIES FORWARD

- Everything from M3: feature engineering, evaluation, workflows
- TrainingPipeline for unsupervised models
- WorkflowBuilder for orchestration
- DriftMonitor for production
- SHAP for interpretation

MODULE 3 COMPLETE

SUPERVISED MACHINE LEARNING FOR BUILDING AND DEPLOYING MODELS

MLFP — ML Foundations for Professionals — Terrene Open Academy

"You now know how to train every major supervised model, evaluate it honestly, interpret its decisions, and deploy it to production. The question is no longer 'can you build ML?' — it is 'what problem will you solve?'"